Defining environment variables

Generally, defining environment variables is OS-dependent, but programming languages have now abstracted away such differences using development packages like Python's *dotenv (.env)*. A *dotenv* file can manage environment variables across dev environments, allowing the developer to use locally managed configuration files. After abstracting the configuration of code to a different *env* file, the code loads all variables from an *env* file.

```
DB_NAME=test_db
DB_USER=username
DB_PASSWORD=password
DB_DOMAIN=localhost
DB_PORT=3000
```

Figure 1: Abstracting the configuration to a *dotenv* file

The code is modified and MongoDB now uses the environment variables to connect to itself, making the code run in different places with a custom *dotenv* file.

```
// Parse .env using a package like dotenv
loadVariableFromEnv()

server = Server()
mongoDB = new MongoDb(
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  process.env.DB_HOST,
  process.env.DB_PORT,
  Process.env.DB_NAME
))
server.get('/users', function() {
mongoDB.getUsers()
})
server.start(port=process.env.PORT)
```

Figure 2: Modifying code to use *dotenv*

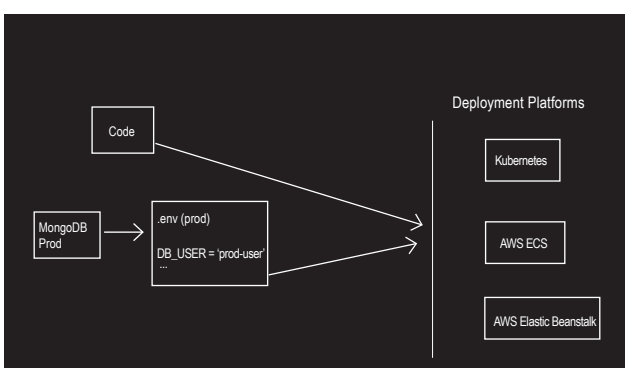

Figure 3: The variables for production environments are fed manually into the deployment infra

Deploying code to production

When the code is to be shifted to production, where you have a MongoDB production instance, you have to compose the *dotenv* file and copy the credentials of MongoDB to it. Similarly, you need to copy all the environment variables and send them to the deployment infrastructure. For example, the developer using Kubernetes will somehow have to set all the environment variables in it, and configure maps and secrets. For instance, an AWS ECS requires you to set that configuration in Fargate, and AWS Elastic Beanstalk has its own console for you to copy the environment variables.

The variables for production environments are fed manually to the deployment infra, which is awful. For every single app built, there is a manual process to copy variables into the deployment infrastructure. This is not a one-time process — every time a developer adds or deletes an environmental variable, it has to be further communicated to the DevOps team, and the latter either adds the variable or removes it from the production system. This is a slow manual cycle, and handling multiple apps with multiple environments

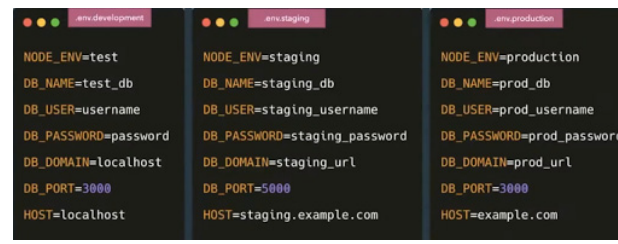

Figure 4: Creating a *dotenv* file per environment

```
if (env is production) {
  config= loadFrom(.env.production)
} else if (env is staging) {
  config = loadFrom(.env.staging)
} else {
  config = loadFrom(.env.development)
}

server = Server()
mongoDB = new MongoDb(
  config.DB_USER,
  config.DB_PASSWORD,
  config.DB_HOST,
  config.env.DB_PORT,
  config.env.DB_NAME
))

server.get('/users', function() {
mongoDB.getUsers()
})

server.start(port=config.PORt)
```

Figure 5: Modifying code to get environment-specific configurations